MFE FORMWORK MR11 SYSTEM

Source Code Repository Guide

Version 1.0

Prepared for

MFE Formwork

Prepared by

MyIntellX project team

Date

21 October 2026

Status

Draft for client review

Document control

Item	Detail
Document	Source Code Repository Guide
System	MFE Formwork MR11 System, version 1.0
Document version	1.0
Date	21 October 2026
Status	Draft for client review
Based on	Repository izzatmyintellx05/MFE, branch main, commit 0cc8c36
Prepared for	MFE Formwork
Prepared by	MyIntellX project team
Intended readers	Client IT, future developers
Classification	Confidential, for client and vendor use only

Revision history

Version	Date	Description	Author
1.0	21 October 2026	First issue for client review	MyIntellX project team

Contents

1. Introduction 3
2. Repository facts 3
3. Branches 3
4. Folder structure 4
5. Where to make common changes 4
6. Build and run commands 5
7. How the code is written 5
8. How to make a change 6
9. Tidying up before handover 6
10. Handing over access 6

1. Introduction

This guide describes the Git repository that holds all of the system's source code: where it is, how it is organised, how to build it, and what should be tidied up before it is handed over. Developers looking for an explanation of the code itself, file by file, should read the three Code Guides (documents 12, 13 and 14).

One point needs attention straight away. The repository is currently public. It should be made private before handover, because it contains sample department workbooks and some infrastructure identifiers (see section 9).

2. Repository facts

Item	Detail
Location	github.com/izzatmyintellx05/MFE
Visibility	Public, and should be changed to private
Default branch	`main`
Release commit	`0cc8c36`, "Department sheets: day headers saved as Excel date numbers show as dates" (15 October 2026)
History	74 commits from 30 September 2026 to 15 October 2026, by two contributors
Tracked files	120
Language	TypeScript for both the frontend and the backend, plus SQL schema files
Licence file	None; ownership and licence terms are set by the contract

3. Branches

Branch	Status	What we recommend
`main`	The release branch; Vercel deploys from it	Protect it, so that merging needs a pull request and a passing build
`client-testing-prep`	Fully merged into `main` (last commit 9 October 2026)	Delete it
`ceo-dashboard`	Fully merged into `main` (last commit 6 October 2026)	Delete it
`claude/tender-sagan-64gqvt`	Has one commit that is not in `main` (30 September 2026) and is 72 commits behind	Review it, then delete it
`azure-deploy`	Mentioned in `azure-pipelines.yml` but does not exist yet	Create it only if Azure DevOps deployment is used

4. Folder structure

Path	What it contains
`src/`	The React frontend
`src/pages/`	The screens: Login, Mr11Dashboard, CeoDashboard, DepartmentPage, PlanningPage, ProductionPage and AdminPage
`src/components/`	`layout/` (the sidebar), `workbook/` (the MR11 table, its side panel, the workbook viewer and the history tables) and `common/` (the market cards, zoom controls and password dialog)
`src/store/`, `src/api/` and `src/utils/`	Sign-in state, the API client, access rules, CEO figures, MR11 filters and number formatting
`server/`	The Express backend
`server/modules/`	One folder for each area of the API (auth, departments, mr11, admin and visualization), each with a routes file and a controller
`server/modules/mr11/`	`mr11.engine.ts` (the consolidation, about 2,200 lines), `mr11.pricing.ts` (prices), and the controller and routes
`server/config/`	`mr11.config.ts` (which columns map where, and in what order) and `database.config.ts`
`server/db/`	`supabase.ts` (database reads and writes) and `prisma.ts` (the in-memory store)
`server/utils/`	The Excel reader, exchange rates (`fx.ts`), the LME price (`lme.ts`) and file storage
`server/middleware/`	The token check and the role check
`api/index.js`	The bundled serverless function for Vercel, produced by `npm run build`
`supabase/schema.sql`	The database tables and seed data for Supabase
`prisma/`	The Prisma schema, its migration and a seed script (the seed only fills the in-memory store)
`scripts/`	Local PostgreSQL control, the end-to-end persistence test and Supabase seeding helpers
`uploads/`	16 sample department workbooks (2 MB) that are loaded in demonstration mode
`public/`	The logo and icons
`docs/`	Earlier design and deployment notes, now replaced by this documentation set
Top level	`server.ts` (the server entry point), `vite.config.ts`, `tsconfig.json`, `vercel.json`, `Dockerfile`, `azure-pipelines.yml`, `azuredeploy.json`, `deploy-azure.sh` and `.env.example`

5. Where to make common changes

The Code Guides explain each of these files in detail.

To change	Look in
Which department column feeds an MR11 column, and the column order	`server/config/mr11.config.ts`
Matching, dates, the monthly forecast, sorting and merging	`server/modules/mr11/mr11.engine.ts`
Price formulas and currency names	`server/modules/mr11/mr11.pricing.ts`
The layout of the Excel export	`exportMr11ToExcel` in `server/modules/mr11/mr11.controller.ts`
Who can open which page	`src/utils/access.ts` on the frontend, and the routes' middleware on the server
The MR11 filters	`src/utils/mr11Filters.ts`
The CEO Executive Hub figures	`src/utils/ceoMetrics.ts` and `src/pages/CeoDashboard.tsx`
Database reads and writes	`server/db/supabase.ts`

6. Build and run commands

Command	What it does
`npm install`	Installs the dependencies; `.npmrc` sets `legacy-peer-deps=true`, which this project needs
`npm run dev`	Starts the development server on port 3000, with the API and frontend together
`npm run lint`	Runs the TypeScript type check
`npm run build`	Builds the frontend into `dist/` and the serverless API into `api/index.js`
`npm start`	Runs the server, in production mode when `NODE_ENV=production` and `dist/` exists
`npm run db:start`, `db:stop` and `db:status`	Control a local PostgreSQL on port 5433
`npm run test:persistence`	Builds the project, then checks that uploads, users and MR11 history survive restarts, using a local database

We confirmed on 21 October 2026, at commit `0cc8c36`, that `npm install`, `npm run lint` and `npm run build` all complete successfully.

7. How the code is written

- The project uses ES modules throughout (`"type": "module"`), and in development TypeScript runs directly through `tsx`.
- The backend is organised by feature. Routes declare who may call them, controllers handle the requests, and the engine and services hold the business rules.
- The `@prisma/client` import points to the in-memory store in `server/db/prisma.ts`, set up in `tsconfig.json`. There is no generated Prisma client when the system runs.

- Comments explain the business rules in plain language, right next to the code that applies them.
- There are no unit tests. Quality is checked through the type check, the build and the persistence test script.

8. How to make a change

1. Create a branch from `main`.
2. Make the change, then run `npm run lint` and `npm run build`.
3. Push the branch and open a pull request, and Vercel will build a preview deployment.
4. Test the preview with real workbooks against a staging database.
5. Merge into `main`, and Vercel will deploy to production.
6. Add the change to the Release Notes.

9. Tidying up before handover

☐ Make the repository private, or move it to the client's GitHub organisation.

☐ Remove the sample workbooks in `uploads/` if they contain real client data. They also remain in the Git history, so if necessary rewrite the history or start a clean repository.

☐ Remove the Supabase project reference and the Azure subscription and tenant ids from `.env.example`, `docs/` and `azure-pipelines.yml`.

☐ Replace the `JWT_SECRET` written in `azure-pipelines.yml` with a pipeline secret.

☐ Correct the default password in the README, or remove the default credentials from it altogether.

☐ Delete the merged branches and protect `main`.

☐ After confirming they are not needed, remove the unused files: `src/pages/Register.tsx`, `src/pages/PendingAccess.tsx` and `src/pages/VisualizationDashboard.tsx` (none of them is routed), `server/server.ts` (an unused entry point) and `server/db/prisma.js` (a compiled copy).

☐ Decide whether `api/index.js` should stay in the repository, since Vercel rebuilds it on every deployment.

☐ Add the client's administrators as repository owners, and keep a record of who has access.

10. Handing over access

System	Held today by	To be handed to
GitHub repository	The vendor account izzatmyintellx05	The client's GitHub organisation or named client administrators
Vercel project	The vendor	A client team account
Supabase project	The vendor	The client's organisation
Azure subscription, if used	The vendor's tenant	The client's tenant

We still need to confirm with the client who should own each account after handover.